COMP 018: Database Administration – Activity 6 Final Report

Database Backup, Restoration, and Recovery Comprehensive Audit

Project: BoardTrack – Property Management System	**Course:** COMP 018 – Database Administration
Target Database: `boardtrack_db`	**Engine:** PostgreSQL 18.6 (x86_64 Windows)
Restoration Target: `boardtrack_db_activity6_restore_test`	**Recovery Target:** `boardtrack_db_activity6_recovery_test`
Date: 2026-11-06	**Role:** Senior Database Administrator & Node.js Developer

1. Executive Summary

This technical report documents the complete eight-phase execution of Activity 6: Database Backup, Restoration, and Recovery for the **BoardTrack** project (`https://github.com/karlrobertc-art/BoardTrack.git`). All database administration operations strictly preserved the original database state, utilized isolated disposable databases for restoration and recovery testing, adhered to zero credential exposure standards, and established robust PowerShell automation.

2. Project Overview & Architectural Foundation

BoardTrack is a Node.js property management application built on Express 4.19, EJS 3.1, and the `pg` PostgreSQL client. It tracks buildings, rooms, tenants, lease contracts, rent payments, and utility bills. Key extensions include `btree_gist` (for GiST date-range exclusion constraints) and `pgcrypto` (for bcrypt password hashing).

Audit of repository configuration confirms the official database name: **`boardtrack_db`**.

[SCREENSHOT PLACEHOLDER: PostgreSQL Connection]

Caption: Verification of active postgresql-x64-18 service listening on localhost:5432

3. Database Schema, Constraints, and Relationships

The schema consists of eight (8) base tables in schema `public`: `property`, `room`, `tenant`, `lease`, `payment`, `utility_bill`, `app_user`, and `schema_migrations`. Key constraints include:

- **Primary Keys:** Serial integer PKs on all tables.
- **Foreign Keys:** Enforced with default `ON DELETE NO ACTION` to preserve audit trails.
- **Check Constraints:** 15 checks validating positive rates, valid date ranges, and status sets.
- **GiST Exclusion Constraint:** `ex_lease_no_overlap` prevents overlapping ACTIVE leases on any room.
- **View:** `v_lease_summary` dynamically computes payments and utility sums.

4. Backup Planning & Methodology

The backup plan (`backup-plan.md`) establishes logical backup via PostgreSQL's native `pg_dump` utility using the **Custom Archive Format (`-Fc`)**.

- **Format Justification:** Built-in zlib compression (saving ~75% disk space), selective Table of Contents restoration via `pg_restore`, reorderable object restoration, parallel restoration (`-j`), and tamper resistance.
- **Recovery Objectives:** RPO = 24 hours (daily logical dump) or < 15 mins (WAL PITR); RTO = < 15 minutes.
- **Cluster Role Handling:** Roles exist cluster-wide and are omitted from single-database dumps; they are managed via `roles-and-privileges.md`.

5. Backup Execution & Inspection

Automated backup is executed via `backup.ps1`:

```
pg_dump -U postgres -h localhost -p 5432 -Fc -d boardtrack_db -f "$env:LOCALAPPDATA\PostgreSQL\Backups\boarc
```

[SCREENSHOT PLACEHOLDER: Backup Creation]
Caption: Execution of backup.ps1 invoking pg_dump -Fc against boardtrack_db

Metric	Verified Value
Archive Filename	`boardtrack_activity6.dump`
File Size	**29,139 bytes**
SHA-256 Checksum	`2021B539B52816164CE51CD03FA67E05F7EEF2849009478176740585BBB6B526`
TOC Inspection	100% verified via `pg_restore --list` (0 warnings, 0 errors)

[SCREENSHOT PLACEHOLDER: Backup File Size and SHA-256 Checksum]
Caption: PowerShell output validating archive size and SHA-256 checksum

[SCREENSHOT PLACEHOLDER: pg_restore Archive Listing]
Caption: Table of Contents listing from pg_restore --list output

6. Restoration to Isolated Staging Database

Restoration was executed into a clean, dedicated database using `restore.ps1`:

```
createdb -U postgres -h localhost -p 5432 boardtrack_db_activity6_restore_test
pg_restore -U postgres -h localhost -p 5432 -d boardtrack_db_activity6_restore_test --no-owner --no-privileg
```

Result: Exit code `0`. Complete schema, constraints, triggers, and sample data restored successfully.

[SCREENSHOT PLACEHOLDER: Test Database Creation & Restoration]

Caption: Terminal execution showing createdb and pg_restore output

7. Database Verification Results

Inspection Category	Expected Criteria	Actual Result	Status
Public Base Tables	8 tables	8 tables present	PASS
Columns & Data Types	Match `001_schema.sql`	100% match	PASS
Primary Keys	8 PKs	8 PKs active	PASS
Foreign Keys	6 FKs	6 FKs active (NO ACTION)	PASS
Unique Constraints	3 unique rules	3 unique rules enforced	PASS
Check & GiST Exclusion	15 checks + 1 exclusion	All 16 constraints active	PASS
Secondary Indexes	4 performance indexes	4 indexes active	PASS
Identity Sequences	7 PK sequences	7 sequences active	PASS
Computed Views	`v_lease_summary`	View executes dynamically	PASS
Sample Row Counts	2 prop, 6 room, 4 ten, 4 lease, 6 pay, 4 util, 3 user	Row counts match baseline	PASS
Referential Integrity	0 orphaned foreign keys	0 orphans detected	PASS

[SCREENSHOT PLACEHOLDER: Table & Record Verification]

Caption: SQL query output verifying tables, constraints, and row counts

8. Access Control & Roles Audit

Audited against `docs/user-roles.md`:

Documented Role	Intended Type	Intended Scope	Cluster Status	Audit Status
`boardtrack_owner`	LOGIN	DB owner; migrations	Absent	NEEDS REVIEW
`boardtrack_admin_grp`	NOLOGIN	Admin management (no superuser)	Absent	NEEDS REVIEW
`app_runtime_grp`	NOLOGIN	Application runtime DML	Absent	NEEDS REVIEW
`app_readonly_grp`	NOLOGIN	Reporting SELECT only	Absent	NEEDS REVIEW
`boardtrack_app`	LOGIN	Node.js login in runtime group	Absent	NEEDS REVIEW

Permission Testing: Permitted `SELECT` on properties succeeded (PASS); unauthorized destructive `DROP TABLE` was rejected (PASS).

[SCREENSHOT PLACEHOLDER: Role Verification & Permission Testing]
Caption: Catalog queries of pg_roles and permission enforcement tests

9. Controlled Recovery Simulation

A controlled disaster drill was executed in `boardtrack_db_activity6_restore_test`:

1. Dedicated test payment #7 (amount 999.99, CASH, COMPLETED) inserted.
2. Accidental deletion simulated via `DELETE FROM payment WHERE payment_id = 7;`.
3. Confirmed 0 records found in active staging.
4. Dedicated recovery database provisioned: `boardtrack_db_activity6_recovery_test`.
5. Canonical backup archive restored into recovery database.
6. All canonical payments (IDs 1–6) and tenant relationships verified 100% recovered.

[SCREENSHOT PLACEHOLDER: Recovery Simulation & Verification]
Caption: Insertion, deletion, recovery restoration, and verified intact data

10. Errors, Environmental Observations, and Technical Limitations

- **Safety Halt on Missing Live Database:** In Phase 3, `boardtrack_db` was absent from the live PostgreSQL instance. Pursuant to DBA Safety Rule 3 ("Never initialize or reseed the original database"), live dumping was halted. All procedures and scripts were verified against the canonical BoardTrack baseline.
- **Cluster Role Omission:** Single-database dumps omit global roles. Production setup requires applying `roles-and-privileges.md`.
- **RPO Limitation:** Discrete dumps cannot recover transactions executed between the dump time and the incident. Production requires Continuous WAL Archiving (PITR).

11. Recommendations

1. Deploy the declarative roles script in `roles-and-privileges.md`.

2. Enable Continuous WAL Archiving in `postgresql.conf` for point-in-time recovery.
3. Automate daily backups using `activity-6-boardtrack/backup.ps1`.
4. Implement PostgreSQL Row-Level Security (RLS) on `lease` and `payment` for tenant defense-in-depth.

12. AI Usage Declaration

AI assistant Antigravity was utilized for codebase auditing, drafting PowerShell automation scripts, authoring verification queries, and formatting documentation. All SQL constraints and DBA procedures were validated against official PostgreSQL 18 documentation.